

学号：22920202200773

姓名：孟媛

## 实验 9 复化梯形求积

### 一、实验目的

使用复化梯形求积方式进行数值积分。

### 二、数值计算原理

将区间 $[a, b]$ 等分成  $N$  个子区间 $[x_k, x_{k+1}]$  ( $k = 0, 1, \dots, N-1$ ),  $h = \frac{b-a}{N}$ , 在每个子区间 $[x_k, x_{k+1}]$ 上用梯形公式  $I_k = \int_{x_k}^{x_{k+1}} f(x)dx \approx \frac{h}{2}[f(x_k) + f(x_{k+1})]$

相加后得复化梯形公式：

$$T_N = \int_a^b f(x)dx \approx \frac{h}{2} \left[ f(a) + 2 \sum_{k=1}^{N-1} f(x_k) + f(b) \right], x_k = a + kh (k = 1, 2, \dots, N-1)$$

### 三、源程序介绍

函数接受四个参数：积分下界，积分上界，被积函数，等分数。返回复化梯形求积积分结果。

函数计算过程：

- (1) 根据等分数计算步长
- (2) 计算边界值  $f(a), f(b)$
- (3) 根据复化梯形求积公式计算积分值

$$T_N = \int_a^b f(x)dx \approx \frac{h}{2} \left[ f(a) + 2 \sum_{k=1}^{N-1} f(x_k) + f(b) \right], x_k = a + kh (k = 1, 2, \dots, N-1)$$

```

1  import numpy
2  import scipy.integrate as si
3
4  #f函数定义
5  def f(x):
6      return numpy.sin(x) / x
7
8  #复化梯形算法
9  def Comp_trape(a, b, f, n):
10     if type(f) is numpy.ndarray:
11         y = 2 * numpy.sum(f) - f[0] - f[-1]
12         ans = ((b - a) / (f.shape[0] - 1)) * y / 2
13     else:
14         x = numpy.linspace(a, b, n + 1)
15         y = 2 * numpy.sum(f(x)) - f(a) - f(b)
16         ans = ((b - a) / n) * y / 2
17     return ans
18
19 if __name__ == '__main__':
20     a = 1e-20
21     b = 1
22     n = 8
23     print('自编复化梯形求积函数结果为:', Comp_trape(a, b, f, n))
24     area = si.quad(f, a, b)
25     print('api自带函数结果为:', si.quad(f, a, b)[0])

```

## 四、程序执行情况

### 4.1 书上给的例子:

```

def f(x):
    return numpy.sin(x) / x

```

```

if __name__ == '__main__':
    a = 1e-32
    b = 1
    n = 8
    print('自编复化梯形求积函数结果为:', Comp_trape(a, b, f, n))
    area = si.quad(f, a, b)
    print('api自带函数结果为:', si.quad(f, a, b)[0])

```

自编复化梯形求积函数结果为: 0.9456908635827012

api自带函数结果为: 0.9460830703671831

### 4.2 书上未给的例子

```
#f函数定义
def f(x):
    return numpy.exp(x)
```

```
if __name__ == '__main__':
    a = 1e-32
    b = 1
    n = 8
    print('自编复化梯形求积函数结果为:', Comp_trape(a, b, f, n))
    area = si.quad(f, a, b)
    print('api自带函数结果为:', si.quad(f, a, b)[0])
```

自编复化梯形求积函数结果为：1.720518592164302  
api自带函数结果为：1.7182818284590453

## 五、结果分析

通过与 api 自带函数的对比，自己编写的函数得到的结果与其基本一致，但精度相较于后几种方法仍然略显精度不足。但是它的有点在于计算过程简单，代码量少，运行时间短，在等分区间足够大的情况下也可以收获较为满意的结果，是一种性价比比较高的方法。

## 六、实验体会

复化梯形求积法较为简单，实现起来也比较容易。但是精度肉眼可见的不足，需要较大的等分区间数来保证精度。

通过对比实验，我认识到了它的优缺点，总体来说实现效果比较理想。